

modifying the same global variable *x*. As an exercise, try to work out how the two compilers obtained their respective outputs by referring to the previous examples. Figure 3 shows the exact output obtained with the gcc and Clang compilers, but look at this only if you fail to get the correct output for both the compilers.

The program *pgm3.c* was also tested with the tcc compiler and the behaviour was similar to that of the Clang compiler. This program was also renamed into a C++ program as *pgm3.cc* and tested with the compilers g++ and Clang++. Just like the previous program, the output of g++ was the same as that of gcc and the output of Clang++ was similar to that of Clang.

Even though the title of the article says ‘Unspecified behaviour in C and C++’, you should not think that C and C++ have common undefined and unspecified behaviours. To better understand this point, consider the program *pgm4.cc* shown below, in which the same line of code *int test = &x > &y;* causes unspecified behaviour in C++ and undefined behaviour in C.

```
#include <stdio.h>

int main()
{
    int x;
    int y;
    int test = &x > &y;
    if(test==0)
    {
        printf("\ng++\n\n");
    }
    else
    {
        printf("\nclang++\n\n");
    }
}
```

In C11, a comparison of pointers to objects is defined only if the pointers point to members of the same object or elements of the same array, whereas in C++14, this is just an unspecified behaviour. The program *pgm4.cc* is tested with the C++ compilers g++ and Clang++, and the output is shown in Figure 4.

As you can observe from the figure, g++ and Clang++ compilers behave differently again. Based on this program, we must understand two important points. First, similar programming language constructs in C and C++ need not be defined similarly in the standards. The example above shows a behaviour undefined in C and unspecified in C++. If you go through the standards of C and C++ carefully, you will be able to identify many such differences in definitions of similar programming language constructs. The second point to remember is that the same behaviour undefined in

```
# gcc pgm3.c
# ./a.out

Sum = 434
# clang pgm3.c
# ./a.out

Sum = 201
```

Figure 3: Different numbers as output

```
# g++ pgm4.cc
# ./a.out

g++
# clang++ pgm4.cc
# ./a.out

clang++
```

Figure 4: A C++ example

one particular standard of a programming language might be strictly defined in a later standard. A careful analysis of C99 and C11 standards will help you find out many examples for such changes in definitions.

Implementation-defined behaviour

There is a sub-category of unspecified behaviour called implementation-defined behaviour. The C11 standard defines implementation-defined behaviour as “unspecified behaviour, where each implementation documents how the choice is made.” There aren’t any details regarding unspecified behaviour and implementation-defined behaviour in the programming language standard, but there are details regarding implementation-defined behaviour in the compiler implementation manual. So, basically, the difference is that the programmer is responsible for figuring out unspecified behaviour all by himself, whereas he will get directions from the compiler implementation manual regarding implementation-defined behaviour. According to the C11 standard, the alternative manner in which the main function may be defined, the number of significant initial characters in an identifier, the number of bits in a byte, the value of a string literal containing a multi-byte character not represented in the execution character set, the accuracy of the floating-point operations, the result of converting a pointer to an integer or vice versa, etc, are examples of implementation-defined behaviour.

I have only mentioned a few examples in each category discussed in this article, but the list is not exhaustive. There is a wide range of undefined, unspecified, and implementation-defined behaviour in C and C++. Due to the many reasons and problems stated earlier, you should refrain from the use of such code in your programs. My advice is that if your code looks crazier than usual, you should refer to the programming language standard to make sure that the code will not exhibit undefined, unspecified, or implementation-defined behaviour. **END** 🐧

 By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science, ns-2 and ns-3. He maintains a technical blog at www.computingforbeginners.blogspot.in. The author can be reached at deepumb@hotmail.com.